

Introduction to Quantum Complexity Theory, Part 1

Quantum Algorithms Decal

Covered:

1. What is Complexity Theory?
2. Classical Probabilistic Computing (BPP)
3. How is Quantum Computation Different?

Central Question: Why are quantum algorithms faster?

What is Complexity Theory?

Complexity Theory studies computational efficiency, such as:

- How much **time** does it take to solve a problem?
- How much **space** (memory) do we need?
- What problems are **fundamentally hard**?

Example Questions:

- Can we sort n numbers faster than $O(n \log(n))$ comparisons?
- Can we solve SAT faster than brute force?
- $P=NP$? $P=BPP$? $P=PSPACE$?

Classical Complexity Classes:

P	Efficiently solvable (polynomial time)
NP	Efficiently verifiable
BPP	Efficiently solvable with randomness
PSPACE	Solvable with polynomial space

Hierarchy:

$$P \subseteq BPP \subseteq? \subseteq PSPACE$$

$$P \subseteq NP \subseteq PSPACE$$

BQP = “Bounded-error Quantum Polynomial time”

Problems solvable efficiently on a quantum computer (quantum analog of BPP)

Theorem 20.17

$$\text{BPP} \subseteq \text{BQP}$$

Quantum computers can do at least everything classical randomized computers can do.

But can they do MORE?

Quantum Algorithms Show Real Speedup

Problem	Classical	Quantum
Integer Factoring (Shor, 1994)	$\exp(n^{1/3})$ years	$O(n^3)$ hours
Unstructured Search (Grover, 1996)	$O(N)$ Check all	$O(\sqrt{N})$ Check $\sqrt{N}$

Impact:

- Breaks RSA encryption (factoring)
- Speeds up database search, optimization

WHY?

Why are quantum algorithms faster?

Common answer: “Quantum computers use quantum randomness!”

WHY?

Why are quantum algorithms faster?

Common answer: “Quantum computers use quantum randomness!”

Is this right?

Quantum algorithms are faster because of fancy randomness

Feynman Quote (1982)

“The only difference between a probabilistic classical world and the equations of the quantum world is that somehow or other it appears as if the probabilities would have to go **negative**.”

Preview: The secret is **interference**, not randomness

Classical Probabilistic Computing

Does Randomness Help?

Example: Polynomial Identity Testing

Problem: Are polynomials $p(x)$ and $q(x)$ equal?

- **Naive:** Expand and compare (exponential time)
- **Randomized:** Pick random r , check if $p(r) = q(r)$
 - If equal: probably identical
 - If different: definitely not identical
 - **Time:** Polynomial!

Insight: Randomness can give efficient algorithms

The Class BPP

BPP = Bounded-error Probabilistic Polynomial time

Definition

A problem is in **BPP** if there's a polynomial-time randomized algorithm that:

- If answer is YES: outputs YES with probability $\geq 2/3$
- If answer is NO: outputs NO with probability $\geq 1/3$

Notes:

- Can amplify to 99.99%+ by repetition
- $2/3$ is arbitrary (any constant $> 1/2$ works)
- **Key constraint:** polynomial time

Example of a Probabilistic Computer

An m -bit register's state = vector $\mathbf{v} \in \mathbb{R}^{2^m}$

Notation:

- $|x\rangle$ = basis vector for definite state x
- $\mathbf{v} = \sum_x v_x |x\rangle$ where $v_x = \text{Pr}[\text{outcome is } x]$

Constraints:

1. $v_x \geq 0$ for all x (probabilities are non-negative)
2. $\sum_x v_x = 1$ (probabilities sum to 1)

Example (2 bits):

$$\mathbf{v} = \frac{1}{2} |00\rangle + \frac{1}{4} |01\rangle + \frac{1}{8} |10\rangle + \frac{1}{8} |11\rangle$$

Measurement: Read register $\rightarrow$ get x with probability v_x

Probabilistic Computation as Linear Algebra

Instead of: “My program tosses coins and does stuff”

Think: “My system is in a probability distribution that evolves”

Initial: $|x\rangle$ (certain state)
↓ [coin toss]
 $p|0\rangle + (1-p)|1\rangle$ (probabilistic state)
↓ [more operations]
Final distribution

Operations as Matrices

An operation = function $F : \mathbb{R}^{2^m} \rightarrow \mathbb{R}^{2^m}$

Two key properties:

1. **Linear:** $F(\sum v_x |x\rangle) = \sum v_x F(|x\rangle)$
2. **Stochastic:** Preserves probability distributions

Matrix representation:

- Each column sums to 1
- All entries ≥ 0

Example: Coin toss operation (1 bit)

$$F = \begin{pmatrix} 1/2 & 1/2 \\ 1/2 & 1/2 \end{pmatrix}$$

$$F(|0\rangle) = \frac{1}{2} |0\rangle + \frac{1}{2} |1\rangle$$

The Classical Framework

Component	Description
State	$\mathbf{v} \in \mathbb{R}^{2^m}$, $v_x \geq 0$, $\sum v_x = 1$
Operations	Linear + stochastic matrices
Measurement	Get x with $\text{Pr} = v_x$
Computation	Sequence of operations

Why is Quantum Computation different?

Quantum States: Almost the Same!

Classical Probabilistic	Quantum
State: $\mathbf{v} \in \mathbb{R}^{2^m}$	State: $\mathbf{v} \in \mathbb{C}^{2^m}$
Constraint: $v_x \geq 0, \sum v_x = 1$	Constraint: $\sum v_x ^2 = 1$
Measurement: $\Pr[x] = v_x$	Measurement: $\Pr[x] = v_x ^2$
Operations: Stochastic	Operations: Unitary

The key difference: $\mathbb{R}$ vs $\mathbb{C}$ (real vs complex)

But what does this mean??

Amplitudes vs Probabilities

Classical:

- Coefficients v_x are **probabilities**
- Must be ≥ 0
- Can only ADD: $\Pr[A \text{ or } B] = \Pr[A] + \Pr[B]$

Quantum:

- Coefficients v_x are **amplitudes** (complex numbers!)
- Can be negative: $-1/\sqrt{2}$ is valid
- Can be complex: $(1 + i)/2$ is valid
- Can **INTERFERE**: constructive or destructive

$$\text{Probability} = |\text{amplitude}|^2$$

Why Negative Amplitudes Matter

Classical example:

Path 1 to outcome A: probability $1/4$

Path 2 to outcome A: probability $1/4$

Total probability for A: $1/4 + 1/4 = 1/2$

Always increases!

Quantum example:

Path 1 to outcome A: amplitude $+1/2$

Path 2 to outcome A: amplitude $-1/2$

Total amplitude for A: $+1/2 - 1/2 = 0$

Total probability: $|0|^2 = 0$

Can cancel completely!

The Hadamard Gate

A Simple Quantum Operation (1 qubit):

$$H : \begin{cases} |0\rangle \rightarrow \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \\ |1\rangle \rightarrow \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) \end{cases}$$

Note the negative sign!

Example:

Start: $|0\rangle$

After H : $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$

Measure: 50% chance of 0, 50% chance of 1

Looks random... but there's hidden structure!

Hadamard Twice: Interference Revealed

Apply Hadamard twice to $|0\rangle$:

Step 1: $|0\rangle \xrightarrow{H} \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$

Step 2: $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \xrightarrow{H} ???$

Expand:

- $H(|0\rangle)/\sqrt{2} = (|0\rangle + |1\rangle)/2$
- $H(|1\rangle)/\sqrt{2} = (|0\rangle - |1\rangle)/2$

Add them:

$$|0\rangle \text{ term: } 1/2 + 1/2 = 1 \quad (\text{constructive!})$$

$$|1\rangle \text{ term: } 1/2 - 1/2 = 0 \quad (\text{destructive!})$$

Result: $|0\rangle$ with certainty!

The General Principle (Grover search example)

Example sketch of quantum speedup:

1. Superposition

- Create equal superposition over all possibilities
- Like exploring all paths at once

2. Marking

- Use a function to mark the answer
- All paths still present

3. Interference

- Apply carefully chosen operations
- Wrong answers interfere **destructively** (cancel)
- Right answer interferes **constructively** (amplifies)

4. Measurement

- Observe the amplified answer

This is impossible with classical probability!

Theorem 20.17: $BPP \subseteq BQP$

But more importantly:

Classical (BPP)	Quantum (BQP)
Amplitudes ≥ 0	Amplitudes can be < 0
Paths only add	Paths can cancel
No interference	Interference!

Example problems where this matters:

- Simon's problem: Exponential quantum speedup
- Factoring (Shor): Exponential quantum speedup
- Search (Grover): Quadratic speedup

Is BQP Really Bigger Than BPP?

What we know: $\text{BPP} \subseteq \text{BQP}$

What we did *not* prove:

- That this inclusion is **strict**
- Every speedup we've seen (Shor, Grover) is evidence, not a proof that $\text{BPP} \neq \text{BQP}$

Open Problem Is $\text{BPP} = \text{BQP}$?

Unknown – just like P vs. NP, this is unproven

Dequantization: an active research program that takes algorithms once thought to need exponential speedup and finds classical algorithms matching them

- E.g. Tang's algorithm (2018) dequantized a quantum recommendation-system algorithm
- Each success **narrows** the conjectured gap of BPP and BQP

The key insights:

Quantum $\neq$ “fancy randomness”

Quantum = complex amplitudes + interference

Amplitudes can cancel (impossible for probabilities)

This enables computational speedups

Whether it always lets quantum *decide* more efficiently is still open

Quantum Complexity Landscape

$$P \subseteq BPP \subseteq \mathbf{BQP} \subseteq PSPACE$$

$$P \subseteq NP \subseteq PSPACE$$

What we know:

- $BPP \subseteq BQP$ (Theorem 20.17)
- $BQP \subseteq PSPACE$
- Quantum speedups arise from interference, not randomness

Open questions:

- Is $NP \subseteq BQP$? (probably not). Does $BPP=BQP$?

Key Takeaways:

1. Complexity theory studies efficiency
2. Quantum $\neq$ randomness, quantum = interference
3. Negative amplitudes enable cancellation
4. This is the mechanism behind every quantum speedup

Sources:

- Arora & Barak: *Computational Complexity* (Ch 20.3-20.4)